

TASK 01 — REST-Based Inter-Service Communication

Step 1 — Understand service-to-service communication

Inter-Service Communication

When two or more microservices exchange information, it is called **inter-service communication**.

Example:

Order Service

| "Give me user 101 details"



User Service

| User information



Order Service

Each service is an **independent application**.

They can have:

- Separate code
- Separate database
- Separate port
- Separate deployment
- Separate lifecycle
- **Synchronous Communication**

In **synchronous communication**, Service A sends a request to Service B and **waits for the response**.

Example:

Order Service

| HTTP Request

User Service

| HTTP Response

Order Service

For example: GET http://localhost:8081/users/101

User Service might return:

```
{  
  "id": 101,  
  "name": "Madhav",  
  "email": "madhav@example.com"  
}
```

The Order Service waits for this response before continuing.

Real-world example

When you place an order:

Order Service

↓

"Is user 101 valid?"

↓

User Service

↓

"Yes, user exists"

↓

Order Service creates order

Asynchronous Communication

In **asynchronous communication**, the sender doesn't have to wait for an immediate response.

Usually, a message broker is used.

Example:

Order Service

| Message

Message Broker

|

Notification Service

Example:

Order Created

↓

Kafka

↓

Email Service

↓

Send confirmation email

Difference

Synchronous

Waits for response

Usually REST/HTTP

Simple to implement

Service dependency is immediate

Asynchronous

Doesn't need immediate response

Usually Kafka/RabbitMQ/SQS

More distributed

Services can be more loosely coupled

REST-Based Communication

REST stands for:

Representational State Transfer

In microservices, REST APIs commonly use HTTP.

For example, User Service provides:

GET /users/101

Order Service calls:

http://localhost:8081/users/101

The response can be JSON:

```
{  
  "id": 101,  
  "name": "Madhav",  
  "email": "madhav@example.com"  
}
```

Common REST methods:

Method	Purpose	Example
GET	Retrieve data	/users/101
POST	Create data	/users
PUT	Update data	/users/101
DELETE	Delete data	/users/101

Request-Response Model

This is one of the most important concepts in your EPIC.

Suppose Order Service needs user information.

Step 1

Order Service sends a request:

GET http://localhost:8081/users/101

Step 2

User Service receives the request.

Step 3

User Service finds the user.

Step 4

User Service sends the response:

```
{  
  "id": 101,  
  "name": "Madhav",  
  "email": "madhav@example.com"  
}
```

Service Dependency

A **service dependency** means one service depends on another service to complete some operation.

For example:

Order Service

| depends on

User Service

Suppose Order Service receives:

POST /orders

with:

```
{  
  "userId": 101,  
  "product": "Laptop"  
}
```

Before creating the order, it might call:

GET http://localhost:8081/users/101

If User Service responds:

```
{  
  "id": 101,  
  "name": "Madhav"
```

```
}
```

the Order Service can continue.

But if User Service is unavailable:

Order Service

```
| GET /users/101
```

User Service **✗** DOWN

the Order Service may fail or need to handle the failure gracefully.

This is why your EPIC also mentions **Resilience**.

Step 2 — Create Two Services

User Service

Name: User Service

Port: 8081

Its responsibility is managing user information.

Example API:

```
GET /users/101
```

Response:

```
{
```

```
  "id": 101,
```

```
  "name": "Madhav",
```

```
  "email": "madhav@example.com"
```

```
}
```

Order Service

Name: Order Service

Port: 8082

Its responsibility is managing orders.

For example:

GET /orders/1

or:

POST /orders

The important part of this EPIC is that **Order Service communicates with User Service.**

pom.xml

Create: user-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.6</version>
    <relativePath/>
  </parent>
  <groupId>com.example</groupId>
  <artifactId>user-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>user-service</name>
  <description>User Microservice</description>
  <properties>
    <java.version>17</java.version>
```

```
</properties>

<dependencies>

  <!-- Spring Web -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

  </dependency>

  <!-- Optional: Development tools -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-devtools</artifactId>

    <scope>runtime</scope>

    <optional>true</optional>

  </dependency>

  <!-- Testing -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-test</artifactId>

    <scope>test</scope>

  </dependency>

</dependencies>

<build>

  <plugins>

    <plugin>

      <groupId>org.springframework.boot</groupId>

      <artifactId>spring-boot-maven-plugin</artifactId>
```

```
        </plugin>
    </plugins>
</build>
</project>
```

If your Spring Initializr generates a different current 3.x version, you can keep the generated version.

Main Application

File: src/main/java/com/example/userservice/UserServiceApplication.java

```
package com.example.userservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class UserServiceApplication {

    public static void main(String[] args) {

        SpringApplication.run(UserServiceApplication.class, args);

    }

}
```

Create Order Service

Project Name: order-service

Group: com.example

Artifact: order-service

Java: 17

Dependency:

Spring Web

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.6</version>
    <relativePath/>
  </parent>
  <groupId>com.example</groupId>
  <artifactId>order-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>order-service</name>
  <description>Order Microservice</description>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <!-- Spring Web -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- DevTools -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <!-- Testing -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>
```

Main Application

File: OrderServiceApplication.java

```
package com.example.orderservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class OrderServiceApplication {

    public static void main(String[] args) {

        SpringApplication.run(OrderServiceApplication.class, args);

    }

}
```

Step 3 — Create User API

Update your controller URL

In UserController.java, use:

```
@RestController

@RequestMapping("/api/v1/users")

public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {

        this.userService = userService;

    }

    @GetMapping("/{id}")

    public ResponseEntity<User> getUserById(

        @PathVariable Long id) {

        User user = userService.getUserById(id);

        if (user == null) {
```

```
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(user);
}
}
```

The important change from the previous Step 2 code is:

```
@RequestMapping("/api/v1/users")
```

Make sure your User Service has sample data

```
users.add(
    new User(
        101L,
        "John",
        "john@example.com"
    )
);
```

You can add more:

```
users.add(
    new User(
        102L,
        "Rahul",
        "rahul@example.com"
    )
);
```

Make sure the port is 8081

application.properties:

```
spring.application.name=user-service
```

server.port=8081

Start User Service

From the user-service project:

mvn spring-boot:run

You should see something similar to:

Tomcat started on port 8081

Started UserServiceApplication

Test the API

Open Postman.

Select:

GET

Enter:

http://localhost:8081/api/v1/users/101

Click **Send**.

You should get:

```
{  
  "id": 101,  
  "name": "John",  
  "email": "john@example.com"  
}
```

Test an invalid user

Try:

GET http://localhost:8081/api/v1/users/999

Expected:

404 Not Found

Step 4 — Create Order API

Order Model

In your Order Service, create:

model/Order.java

```
package com.example.orderservice.model;
```

```
public class Order {
```

```
    private Long id;
```

```
    private Long userId;
```

```
    private String product;
```

```
    private Integer quantity;
```

```
    private Double amount;
```

```
    private String status;
```

```
    public Order() {
```

```
    }
```

```
    public Order(
```

```
        Long id,
```

```
        Long userId,
```

```
        String product,
```

```
        Integer quantity,
```

```
        Double amount,
```

```
        String status) {
```

```
        this.id = id;
```

```
        this.userId = userId;
```

```
        this.product = product;
```

```
        this.quantity = quantity;
```

```
        this.amount = amount;
```

```
        this.status = status;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public String getProduct() {
        return product;
    }

    public void setProduct(String product) {
        this.product = product;
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }

    public Double getAmount() {
```

```

        return amount;
    }

    public void setAmount(Double amount) {
        this.amount = amount;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}

```

Order Service Class

Create: service/OrderService.java

```

package com.example.orderservice.service;

import com.example.orderservice.model.Order;
import org.springframework.stereotype.Service;

@Service

public class OrderService {

    public Order getOrderById(Long id) {
        if (id.equals(5001L)) {
            return new Order( 5001L,
                101L, "Laptop",2,150000.00,"CONFIRMED");
        }

        return null;
    }
}

```

For now, we are using **in-memory data** to understand the microservice communication.

Step 5 — Implement Service-to-Service Call

Suppose the client wants to get an order:

GET /orders/501

The Order Service knows that order 501 belongs to user 101.

So Order Service needs user information.

Instead of doing this:

Order Service → User Database ❌

it should do:

Order Service → User Service → User Database

Example

Order Service receives:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "amount": 50000,  
  "userId": 101  
}
```

Then Order Service calls:

GET http://localhost:8081/users/101

The User Service responds:

```
{  
  "id": 101,  
  "name": "John",  
  "email": "john@example.com"  
}
```

Order Service then combines both pieces of information.

Step 6 — Implement Using Spring HTTP Client

Common approaches

RestClient

Modern Spring applications can use:

RestClient

Conceptually:

```
User user = restClient
```

```
    .get()  
    .uri("http://localhost:8081/users/{id}", userId)  
    .retrieve()  
    .body(User.class);
```

WebClient

Spring also provides:

WebClient

It is commonly used when you want reactive/non-blocking communication.

Example:

```
User user = webClient
```

```
    .get()  
    .uri("/users/{id}", userId)  
    .retrieve()  
    .bodyToMono(User.class)  
    .block();
```

OpenFeign

Another popular approach is:

@FeignClient

Instead of manually constructing HTTP requests, you define an interface.

For example:

```
@FeignClient(name = "user-service", url = "http://localhost:8081")
```

```
public interface UserClient {  
    @GetMapping("/users/{id}")  
    User getUserById(@PathVariable Long id);  
}
```

Then your Order Service can simply call:

```
User user = userClient.getUserById(order.getUserId());
```

This makes service-to-service communication easier to read.

UserRepository

It would be wrong to copy/use that repository inside Order Service to access the User database.

Wrong architecture

Order Service

|

UserRepository

|

User Database

Correct architecture

Order Service

|

UserClient

| HTTP

User Service

|

UserRepository

|

User Database

The User Service controls:

UserController



UserService



UserRepository



UserDatabase

The Order Service controls:

OrderController



OrderService



OrderRepository



OrderDatabase

And communication happens between the services through APIs.

Step 7 — Create Combined Response

Suppose Order Service has:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "amount": 50000,  
  "userId": 101  
}
```

User Service returns:

```
{  
  "id": 101,
```

```
"name": "John",  
"email": "john@example.com"  
}
```

Instead of returning two separate responses to the client, Order Service creates a **combined response**:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "amount": 50000,  
  "user": {  
    "id": 101,  
    "name": "John",  
    "email": "john@example.com"  
  }  
}
```

This is called **response aggregation** or **data composition**.

Complete Request Flow

Let's understand the complete process step by step.

Step 1 — Client sends request

GET /orders/501

The request goes to Order Service.

Step 2 — Order Controller receives request

Example:

```
@GetMapping("/orders/{id}")  
public OrderResponse getOrder(@PathVariable Long id) {  
    return orderService.getOrder(id);  
}
```

The controller passes the request to the service layer.

Order Service gets order

The Order Service searches its own database.

Example:

Order ID: 501

Product: Laptop

Amount: 50000

User ID: 101

So:

`order.getUserId()`

returns:

101

Order Service calls User Service

Now the Order Service calls:

GET /users/101

The User Service receives the request.

User Service gets user

User Service searches its own database.

It finds:

ID: 101

Name: John

Email: john@example.com

It returns:

```
{  
  "id": 101,  
  "name": "John",  
  "email": "john@example.com"  
}
```

Order Service receives User response

Order Service now has two objects.

Order

Order ID = 501

Product = Laptop

Amount = 50000

User ID = 101

User

ID = 101

Name = John

Email = john@example.com

Order Service combines them

It creates:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "amount": 50000,  
  "user": {  
    "id": 101,  
    "name": "John",  
    "email": "john@example.com"  
  }  
}
```

Step 8 — Test successful communication

Verify that User exists

First, make sure the User Service contains the user.

For example:

GET /users/101

Expected response:

```
{  
  "id": 101,  
  "name": "John",  
  "email": "john@example.com"  
}
```

This proves:

User Service



User 101 exists

If the user doesn't exist, Order Service cannot retrieve the user's information.

Verify that Order exists

Next, make sure the Order Service contains the order.

For example:

GET /orders/501

The Order might contain:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "amount": 50000,  
  "userId": 101  
}
```

Notice this important field:

"userId": 101

This tells the Order Service:

"This order belongs to User 101."

Order Service calls User Service

Now the important part happens.

The Order Service takes:

userId = 101

and calls the User Service:

GET /users/101

The User Service returns:

```
{  
  "id": 101,  
  "name": "John",  
  "email": "john@example.com"  
}
```

Order Service receives User information

The Order Service now has two sets of data.

Order data

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "amount": 50000,  
  "userId": 101  
}
```

User data

```
{  
  "id": 101,  
  "name": "John",  
  "email": "john@example.com"  
}
```

The Order Service combines them.

Final successful response

The client receives:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "amount": 50000,  
  "user": {  
    "id": 101,  
    "name": "John",  
    "email": "john@example.com"  
  }  
}
```

Order Service



User Service



User information



Order Service



Client

is working correctly.

Step 9 — Test Failure

stop User Service

Normally:

Order Service



User Service



User Data

But what happens if User Service crashes?

For example:

Order Service

| HTTP request

User Service 

The Order Service can no longer retrieve user information.

This allows you to test how your application behaves when a dependent microservice is unavailable.

Stop User Service

Suppose your User Service is running.

Stop it.

Example, if you're running the application from IntelliJ IDEA or Eclipse, stop the User Service application.

If running from the terminal, you can stop the running process using the appropriate terminal interruption, commonly:

Ctrl + C

Now:

User Service = DOWN

Call Order API

Now call the Order API again.

For example:

GET /orders/501

The request reaches:

Client

↓

Order Service

The Order Service finds:

userId = 101

So it tries:

GET /users/101

But User Service is stopped.

Therefore:

Order Service

| GET /users/101

User Service

X

NOT RUNNING

failure can occur

Depending on your current implementation, you may see an error such as:

Connection refused

or:

Connection timed out

or an HTTP error returned by your application, such as:

500 Internal Server Error

The exact response depends on how your HTTP client and exception handling are configured.

Step 10 — Document

Client → Order Service

Client sends a request.

GET /orders/501

Order Service → User Service

Order Service needs user information.

GET /users/101

User Service → Order Service

User Service returns:

```
{  
  "id": 101,  
  "name": "John",
```

```
"email": "john@example.com"
}
```

Order Service → Client

Order Service creates the combined response.

```
{
  "orderId": 501,
  "product": "Laptop",
  "amount": 50000,
  "user": {
    "id": 101,
    "name": "John",
    "email": "john@example.com"
  }
}
```

Better Sequence Diagram

we can document it like this:

Client	Order Service	User Service
--- GET /orders/501 -->		
	--- GET /users/101 --->	
	<--- User Details -----	
<-- Combined Response -		

This is a clearer representation.

Detailed Sequence

Client

| GET /orders/501

↓

Order Service

| Find Order 501

Order Database

| Order + userId = 101

Order Service

| GET /users/101

User Service

| Find User 101

User Database

| User Details

User Service

| User JSON

Order Service

| Combine Order + User

Client